PROJECT REPORT

Shared Memory Based Data Exchange System

Operating Systems and Systems Programming

Section	OS Section 13
Team Number	24

Submitted by

Student Name	Student ID
S. Varun Sridhar Reddy	2520090083
Jashwanth Reddy	250080025

A concise technical report describing the design, implementation, execution, and evaluation of a two-way inter-process communication system using shared memory.

1. Abstract

This project demonstrates inter-process communication (IPC) using System V shared memory in C on Linux. A parent process writes a message into a shared memory segment, and a child process reads it and sends a reply through the same shared region. The program illustrates how two processes can exchange data through a common memory area. Status flags coordinate this classroom demonstration, and the program detaches and removes the shared memory segment during cleanup.

2. Introduction

Inter-process communication is an operating-system capability that allows independent processes to exchange information and coordinate their work. Shared memory is an IPC mechanism in which multiple processes attach the same memory segment to their address spaces. Once the segment is established, both processes can access the same data directly. Correct synchronization is important whenever processes access shared data concurrently.

3. Problem Statement

Processes normally execute in separate address spaces. A mechanism is therefore required for one process to share information with another and receive a response. This project addresses that requirement with a small parent–child data exchange system based on System V shared memory.

4. Objectives

- Implement IPC using System V shared memory calls.
- Demonstrate two-way message exchange between parent and child processes.
- Use status flags to indicate when a message or reply is ready.
- Demonstrate process creation with fork() and process waiting with waitpid().
- Detach and remove shared memory after communication.

5. Technologies and Requirements

Component	Purpose
C language	Implements parent and child process logic.
Linux / Ubuntu / WSL	Provides System V IPC and process-management interfaces.
GCC	Compiles the source code.
GNU Make	Builds the program using the Makefile.
System V shared memory	Provides the common region for messages and replies.

6. System Design and Architecture

The program creates and attaches a shared memory segment. The parent stores the initial message before creating the child. After fork(), the child waits for the ready flag, reads the message, and writes a reply. The child sets reply_ready to indicate that the reply is available. The parent reads and displays the reply, waits for the child to finish, detaches the memory, and marks the shared segment for removal.

Parent Process	Shared Memory	Child Process
Writes message Sets ready = 1	message[] reply[] ready / reply_ready	Waits for ready Reads message
Waits for reply_ready Reads reply Cleans up	One common region attached to both processes	Writes reply Sets reply_ready = 1 Exits

7. Implementation Files

main.c: Creates and attaches shared memory, reads the parent message, creates a child using fork(), coordinates message exchange, prints the reply, and performs cleanup.
shared.h: Defines the shared data structure with message and reply buffers and status flags.
Makefile: Provides a repeatable GCC build command.
README.md: Documents the project, build instructions, and expected behavior.

8. Algorithm and Workflow

1. Create a shared memory segment using shmget().
2. Attach it using shmat().
3. Initialize status flags and collect the parent message.
4. Create the child process using fork().
5. The parent sets ready; the child waits for the flag and reads the message.
6. The child writes a reply and sets reply_ready.
7. The parent reads the reply and waits for child termination.
8. Detach from shared memory and remove the segment using shmctl(..., IPC_RMID, ...).

9. Key System Calls and Libraries

Call / Header	Purpose
shmget() — sys/shm.h	Creates or obtains a shared memory segment.
shmat() — sys/shm.h	Attaches the segment to a process address space.
shmdt() — sys/shm.h	Detaches the segment from a process address space.
shmctl() — sys/shm.h	Controls the segment; IPC_RMID marks it for removal.
fork() — unistd.h	Creates the child process.
waitpid() — sys/wait.h	Waits for the child process to terminate.
fgets(), printf(), string.h	Read/display input and trim the newline.

10. Sample Execution

```
Enter message for child: Hello child!

Child received: Hello child!
Child, type your reply: Hello parent!

Parent received child reply: Hello parent!
Communication completed!
```

11. Advantages

• Fast access to shared data after setup.
• Demonstrates a core operating-system IPC mechanism.
• Supports two-way data exchange through one common region.
• Uses a compact C implementation suitable for a laboratory demonstration.

12. Limitations and Synchronization Considerations

This version is an educational demonstration. The ready and reply_ready fields use simple polling; the volatile qualifier alone does not provide robust inter-process synchronization or memory-ordering guarantees. A production implementation should use process-shared semaphores or another appropriate synchronization primitive, together with timeout handling, error checks, and recovery for abnormal process termination.

13. Applications and Future Enhancements

Shared memory can be useful in high-throughput local data pipelines, multimedia processing, telemetry exchange, and communication between cooperating processes. Future enhancements could include process-shared semaphores, a ring buffer for multiple messages, sequence numbers, stronger error handling, and performance comparisons with pipes or sockets.

14. Conclusion

The Shared Memory Based Data Exchange System demonstrates two-way message exchange between a parent and child process through a System V shared memory segment. It combines shared-memory management, process creation, basic coordination, and resource cleanup. The project provides a practical foundation for understanding IPC and can be improved with stronger synchronization and error handling.

15. Team Details

Team	Section	Member	Student ID
24	OS Section 13	S. Varun Sridhar Reddy	2520090083
24	OS Section 13	Jashwanth Reddy	250080025

References

1. Linux manual pages: shmget(2), shmat(2), shmdt(2), shmctl(2), fork(2), and waitpid(2).
2. Linux man-pages project: https://man7.org/linux/man-pages/.
3. GNU Compiler Collection documentation: https://gcc.gnu.org/